

## Continuous State Spaces in RL 🚗

So far, we have looked at simple examples like the Mars Rover where the robot is in one of 6 specific "boxes" ( $s_1, \dots, s_6$ ). This is a **Discrete State Space**.

In the real world, robots (cars, helicopters, drones) don't jump between discrete boxes. They move smoothly through space. Their position is defined by continuous numbers (like 2.5 meters, 10.43 degrees). This is a **Continuous State Space**.

### Example 1: The Self-Driving Truck

Imagine controlling a toy truck. To know its state, you need more than a single integer. You need a vector of continuous values:

- **Position:**  $x, y$  coordinates.
- **Orientation:**  $\theta$  (Angle it is facing, 0-360 degrees).
- **Velocity:**  $\dot{x}, \dot{y}$  (Speed in x and y directions).
- **Angular Velocity:**  $\dot{\theta}$  (How fast it is turning).

**State Vector ( $s$ ):**

$$s = [x, y, \theta, \dot{x}, \dot{y}, \dot{\theta}]$$

This state is a vector of **6 numbers**.

---

### Example 2: The Autonomous Helicopter

A helicopter moves in 3D space, making the state even more complex.

- **Position:**  $x, y, z$  (North/South, East/West, Altitude).
- **Orientation (Angles):**
  - **Roll ( $\phi$ ):** Tilting Left/Right.
  - **Pitch ( $\theta$ ):** Tilting Forward/Backward.
  - **Yaw ( $\omega$ ):** Compass direction (North, East, etc.).
- **Velocities:**  $\dot{x}, \dot{y}, \dot{z}$  (Speed in 3 directions).
- **Angular Velocities:**  $\dot{\phi}, \dot{\theta}, \dot{\omega}$  (Rate of rotation).

**State Vector ( $s$ ):**

$$s = [x, y, z, \phi, \theta, \omega, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\omega}]$$

This state is a vector of **12 continuous numbers**.

---

## The Challenge

In a Discrete State Space (like the 6-state rover), we could just create a table for the Q-function:

- Row 1: Q(State 1, Left)
- ...
- Row 6: Q(State 6, Right)

In a **Continuous State Space**, the state  $s$  is a list of real numbers. Since there are infinite possible combinations of coordinates and speeds, **we cannot use a table**. We need a function that can take these continuous numbers as input and output the Q-value.

## Example Application: The Lunar Lander 🚀

To understand how Reinforcement Learning works in continuous state spaces, we will use the **Lunar Lander** simulation (a popular environment for RL researchers).

### The Objective

You control a spaceship falling toward the moon.

- **Goal:** Fire thrusters to land safely between two yellow flags (the landing pad).
  - **Failure:** Crashing into the surface.
  - **Success:** A soft landing on the pad.
- 

### 1. The Action Space (Discrete)

At every time step, the agent must choose exactly **one** of four possible actions:

1. **Do Nothing:** Let gravity and inertia pull the ship down.
  2. **Fire Left Thruster:** Pushes the lander to the **Right**.
  3. **Fire Main Engine:** Pushes the lander **Up** (against gravity).
  4. **Fire Right Thruster:** Pushes the lander to the **Left**.
- 

### 2. The State Space (Continuous)

The state  $s$  is a vector of **8 numbers** describing the lander's physical situation.

$$s = [x, y, \dot{x}, \dot{y}, \theta, \dot{\theta}, l, r]$$

- **Position:**  $x$  (Horizontal),  $y$  (Vertical).
  - **Velocity:**  $\dot{x}$  (Horizontal speed),  $\dot{y}$  (Vertical speed).
  - **Angle:**  $\theta$  (Tilt),  $\dot{\theta}$  (Angular velocity/spin).
  - **Leg Sensors:**
    - $l$ : Left leg touching ground? (Binary: 0 or 1).
    - $r$ : Right leg touching ground? (Binary: 0 or 1).
- 

### 3. The Reward Function

The designers of this environment carefully crafted the rewards to encourage smooth flying and fuel efficiency.

- **Good Rewards (+):**

- Moving closer to the landing pad.
- **+100** to **+140**: Successfully landing on the pad.
- **+100**: Achieving a "soft landing" (not crashing).
- **+10**: For each leg that touches the ground.
- **Bad Rewards (-):**
  - Moving away from the landing pad.
  - **-100**: Crashing.
  - **-0.3**: Firing the **Main Engine** (Fuel cost).
  - **-0.03**: Firing **Side Thrusters** (Fuel cost).

***Insight:** Designing the Reward Function is an art. It is much easier to specify incentives (e.g., "Don't waste fuel," "Don't crash") than to write a program specifying the exact thruster movements for every millisecond.*

---

## Summary

- **Goal:** Learn a policy  $\pi(s)$  that maps the 8 state variables to one of the 4 actions.
- **Metric:** Maximize the discounted return (using a high discount factor, e.g.,  $\gamma = 0.985$ ).

## Deep Q-Networks (DQN) 🧠

The core idea of DQN is to train a **Neural Network** to approximate the State-Action Value Function,  $Q(s, a)$ . Once we have this network, we can input any state  $s$  and know exactly which action  $a$  yields the highest return.

### The Neural Network Architecture

We design a network to predict the Q-value.

- **Input ( $X$ ):** A vector of **12 numbers**.
    - **State ( $s$ ):** 8 numbers (position, velocity, angle, leg sensors).
    - **Action ( $a$ ):** 4 numbers (One-Hot Encoded vector).
      - *Nothing*:  $[1, 0, 0, 0]$
      - *Left*:  $[0, 1, 0, 0]$
      - *Main*:  $[0, 0, 1, 0]$
      - *Right*:  $[0, 0, 0, 1]$
  - **Hidden Layers:**
    - Layer 1: 64 units.
    - Layer 2: 64 units.
  - **Output ( $Y$ ):** A single number representing  $Q(s, a)$ .
- 

### Creating the Training Set

Since this is Reinforcement Learning, we don't start with a labeled dataset of "correct answers." We have to create our own data by experimenting.

#### 1. Exploration (Replay Buffer):

- The lander flies around (initially randomly).
- We record every step as a **tuple**:  $(s, a, R, s')$ .
  - *Example*: (State A, Fire Main Engine, Reward -0.3, State B).
- We store the most recent **10,000 tuples** in a memory bank called the **Replay Buffer**.

#### 2. Generating Targets ( $Y$ ):

- To train the network, we need an input  $X$  and a target output  $Y$ .
- **Input  $X$ :** The state and action from the tuple  $(s, a)$ .
- **Target  $Y$ :** Calculated using the **Bellman Equation**:
$$Y = R(s) + \gamma \max_{a'} Q(s', a')$$

- *Crucial Note:* Since we don't know the true  $Q$  yet, we use our **current neural network** to estimate the  $Q(s', a')$  part. It's a guess, but it gets better over time.
- 

## The Full DQN Algorithm

1. **Initialize:** Create the Neural Network with random weights (it knows nothing).
  2. **Loop (Repeat forever):**
    - **Act:** Play the game. Choose actions (sometimes random, sometimes using the current network).
    - **Store:** Save the experience  $(s, a, R, s')$  into the **Replay Buffer**.
    - **Train:**
      - Grab a batch of recent experiences from the buffer.
      - Create training examples:  $X = (s, a), Y = R + \gamma \max Q(s', a')$ .
      - Train the network to minimize the error between its prediction and  $Y$  (using Mean Squared Error).
    - **Update:** The network is now slightly smarter. Use this smarter network for the next loop.
- 

## Why This Works

- At first, the network's guesses are garbage.
- However, the **Immediate Reward ( $R$ )** is a fact (ground truth).
- By constantly training the network to match " $R$  + future guess," the truth of  $R$  slowly propagates backward through the states.
- Over thousands of updates, the network eventually learns to accurately predict the  $Q$ -value for every state.

## Optimizing the DQN Architecture ⚡

In the previous video, we discussed a conceptual architecture where the network takes *both* the State and the Action as input. While correct in theory, it is computationally inefficient.

Most practical implementations (including the one you will use in the practice lab) use a modified architecture.

### 1. The "Inefficient" Approach (Previous Version)

- **Input:** State vector ( $s$ ) + Action vector ( $a$ ) = **12 Inputs**.
- **Output:** Single Q-value ( $Q(s, a)$ ).
- **The Problem:** To decide which action to take in the Lunar Lander (which has 4 actions), you have to run the neural network **4 separate times** (once for each action) to compare the results. This is slow.

### 2. The "Efficient" Approach (Standard DQN)

Instead of feeding the action as an *input*, we train the network to output Q-values for **all possible actions simultaneously** based solely on the state.

[Image of neural network architecture]

- **Input:** Only the State vector ( $s$ ) = **8 Inputs**.
- **Hidden Layers:**
  - Layer 1: 64 units.
  - Layer 2: 64 units.
- **Output:** A vector of **4 units**.
  - Unit 1:  $Q(s, \text{Nothing})$
  - Unit 2:  $Q(s, \text{Left})$
  - Unit 3:  $Q(s, \text{Main})$
  - Unit 4:  $Q(s, \text{Right})$

### Why This is Better

1. **Faster Inference:** When the robot needs to act, we run the network **once**. We get all 4 Q-values immediately and simply pick the maximum.  
Best Action =  $\text{argmax}([Q_1, Q_2, Q_3, Q_4])$
2. **Faster Training:** When calculating the target  $Y$  for the Bellman Equation, we need the term  $\max_{a'} Q(s', a')$ . With this architecture, we pass the *next state* ( $s'$ ) into the network once, get all potential future Q-values, and simply take the max.

## The $\epsilon$ -Greedy Policy: Exploration vs. Exploitation

When the learning algorithm is running, the Neural Network's estimates of  $Q(s, a)$  are initially random and likely wrong. This creates a dilemma: **How do we choose actions based on a "brain" that doesn't know anything yet?**

### The Problem with Being "Greedy"

If we *always* choose the action with the highest Q-value (a "Greedy" approach):

1. Suppose the network is initialized such that "Fire Main Engine" has a very low value.
  2. The agent will **never** choose "Fire Main Engine" because it thinks it is bad.
  3. Because it never tries it, it never receives the positive reward for using it.
  4. **Result:** The agent gets stuck in a loop of bad behavior and never discovers the optimal strategy.
- 

### The Solution: $\epsilon$ -Greedy

To solve this, we introduce randomness. We choose a parameter called **Epsilon** ( $\epsilon$ ), which represents the probability of acting randomly.

- **Exploitation (Probability  $1 - \epsilon$ ):**
  - We use what we have learned so far.
  - We pick the action that maximizes  $Q(s, a)$ .
  - *Example:* 95% of the time ( $\epsilon = 0.05$ ).
- **Exploration (Probability  $\epsilon$ ):**
  - We ignore the neural network.
  - We pick an action **completely at random**.
  - *Example:* 5% of the time.

### Why do this?

Random actions force the agent to try things it thinks are "bad." Occasionally, one of these random actions will result in a high reward, allowing the Neural Network to update its Q-values and "realize" that the action is actually good.

---

### Exploration vs. Exploitation Trade-off

This is a fundamental concept in Reinforcement Learning.

- **Exploration:** Trying new things to gather more information (Risk of low reward now, potential for high reward later).

- **Exploitation:** Using current knowledge to maximize the immediate reward (High reward now, no new knowledge gained).

## Refinement: Decaying Epsilon

In practice, we usually change  $\epsilon$  over time.

1. **Start High ( $\epsilon \approx 1.0$ ):** At the beginning, the agent knows nothing. We want it to act randomly to explore the environment thoroughly.
  2. **Decrease Gradually:** As the agent learns, we slowly lower  $\epsilon$ .
  3. **End Low ( $\epsilon \approx 0.01$ ):** Once the agent is trained, we want it to use its learned skills (Exploit) 99% of the time, keeping only a tiny chance of randomness to handle unexpected situations.
- 

## A Note on RL Difficulty

Prof. Ng notes that Reinforcement Learning algorithms are much more "**finicky**" than Supervised Learning algorithms.

- **Supervised Learning:** A bad learning rate might make training 3x slower.
- **Reinforcement Learning:** A bad parameter (like  $\epsilon$ ) might make training **10x or 100x slower**, or prevent learning entirely. Tuning these parameters is often the hardest part of RL.

## 1. Mini-Batches 🚗

**Problem:** Standard gradient descent (using the entire dataset) is slow, especially when the dataset is large.

- In Reinforcement Learning, we might have a Replay Buffer with 100,000 or 1,000,000 experiences.
- Calculating the gradient over *all* these examples every single step is computationally expensive.

**Solution:** Use **Mini-Batch Learning**.

- Instead of using the full 100,000 examples, pick a random subset (e.g., **1,000 examples**) for each training step.
- **Effect:**
  - **Speed:** Each step is much faster (computing gradients for 1,000 items is instant compared to 100,000).
  - **Noise:** The path to the minimum is "noisier" (zig-zagging) because each batch is just an approximation of the whole.
  - **Overall:** It converges *much* faster in terms of wall-clock time.

**In RL Context:**

Instead of training on the *entire* history of the robot's life every time, we just grab a random sample of recent memories (e.g., 64 or 128 experiences) and train on those.

---

## 2. Soft Updates ☁️

**Problem:** In the original algorithm, we completely replaced the old network weights ( $W$ ) with the new weights ( $W_{new}$ ) at every step ( $Q = Q_{new}$ ).

- If  $Q_{new}$  is a bad or noisy estimate (which happens often in RL), overwriting the old network completely can cause the agent to "forget" what it learned or make the training unstable/diverge.

**Solution:** Use **Soft Updates** (Polyak Averaging).

Instead of a 100% replacement, we slowly blend the new weights into the old ones.

**Formula:**

$$W \leftarrow \tau W_{new} + (1 - \tau)W$$

$$B \leftarrow \tau B_{new} + (1 - \tau)B$$

- $\tau$  (**Tau**): A small number (e.g., **0.001** or **0.01**).

- If  $\tau = 0.01$ : We keep **99%** of the old weights and mix in just **1%** of the new learned weights.

**Effect:**

- The network changes **gradually**.
  - This prevents catastrophic forgetting and makes the training process **much more stable** and reliable.
- 

**Summary of the Final DQN Algorithm**

1. **Experience:** Agent plays and stores tuples  $(s, a, R, s')$  in Replay Buffer.
2. **Mini-Batch:** Sample a small batch (e.g., 64) from the buffer.
3. **Train:** Calculate targets ( $Y$ ) and update the **Local Network** weights ( $W_{new}$ ) using Gradient Descent.
4. **Soft Update:** Update the **Target Network** weights ( $W$ ) by blending in a tiny amount of  $W_{new}$  ( $\tau W_{new} + (1 - \tau)W$ ).

# The State of Reinforcement Learning: Hype vs. Reality

Reinforcement Learning (RL) is fascinating—it was even the subject of Andrew Ng's PhD thesis! However, despite the massive research momentum and media attention, there is a significant gap between what is possible in a lab and what is deployed in the real world.

## 1. The "Sim-to-Real" Gap 🎮 vs. 🤖

A major reason for the hype is that most research breakthroughs happen in **Simulated Environments** (like video games or physics engines).

- **The Reality:** It is *much* easier to get an RL algorithm to work in a simulation than on a real physical robot.
- **The Challenge:** In a simulation, the physics are perfect, and data is infinite. In the real world, sensors are noisy, physics are complex, and time is limited.
- **Warning:** If you build a system that works perfectly in a simulation, do not assume it will work immediately in the real world. This transition is notoriously difficult.

## 2. Frequency of Application

While RL gets a lot of headlines (e.g., beating world champions at Go), it is currently the **least used** of the three machine learning pillars in industry.

- **Supervised Learning:** Used constantly (Spam filters, ad clicking, medical diagnosis, speech recognition).
- **Unsupervised Learning:** Used frequently (Recommender systems, clustering, anomaly detection).
- **Reinforcement Learning:** Used sparingly (Robotics, specific control tasks, complex game AI).

**Practical Advice:** In your day-to-day work as a Machine Learning Engineer, you will likely use Supervised and Unsupervised learning far more often than RL.

---

## Conclusion: A Pillar for the Future

Despite these limitations, Reinforcement Learning remains a fundamental pillar of Machine Learning. The research is moving fast, and its potential to solve complex, dynamic problems in the future is immense.

Understanding RL frames your mind to think about **automated decision-making**, which is a valuable skill even if you don't deploy it into production tomorrow.

---

## Final Task: Land the Lunar Lander! 🚀

You now have the theoretical knowledge to complete the final practice lab. You will write the code to train a Deep Q-Network (DQN) that autonomously lands a spacecraft on the moon.